

INTEGRATION GUIDE — Team Producers, Consumer, Spark & Streamlit

Read this end-to-end once, then follow the “Quick Start” checklists. This repo is **Docker-only** (no local venv needed).

0) What you will plug in

Each teammate adds **their own Kafka producer** that pulls from Spotify (possibly different endpoints → different JSON schemas). You will also add an optional **Streamlit tab** for your visuals, and (if needed) extend the **consumer** or **Spark** mapping.

This guide explains exactly **where** to put files, **what** to name them, **how** to configure topics/collections, and **which Makefile targets** to run.

1) Requirements

- Linux or Windows/macOS with **Docker Desktop** (WSL2 on Windows).
- Internet access (to pull images & reach Spotify).
- A valid Spotify API app (client id/secret) — each teammate uses their own creds.
- Optional (Ubuntu): [src/tools/install_docker_ubuntu.sh](#) installs Docker+Compose.

2) Repository layout (what matters for you)

```
.
├── INTEGRATION.md, INTEGRATION.pdf, SETUP.md, STATUS.md
├── src
│   ├── docker-compose.yml
│   ├── makefile                # main Makefile (docker-only workflow)
│   ├── make/
│   │   ├── _template.mk        # template for per-user helpers (optional)
│   │   └── avd.mk              # example user helper file
│   ├── envs/                   # put your per-user .env.<prefix> here
│   ├── producers/              # your producer goes here
│   │   ├── avd_producer.py      # example: Dorin's producer
│   │   └── example_producer.py  # minimal example to copy
│   ├── consumers/
│   │   └── kafka_consumer.py    # Kafka → Mongo writer (generic, works for
all topics)
│   │   ├── Dockerfile, requirements.txt
│   ├── spark/
│   │   ├── streaming_job.py     # Structured Streaming (Kafka → Mongo
aggregation)
│   │   ├── schemas.py          # example schemas/utilities
│   │   └── Dockerfile, requirements.txt
```

```

├── app/                                # Streamlit UI
│   ├── streamlit_app.py                # UI entrypoint
│   ├── tabs/                          # add your own <prefix>.py tab here
│   └── team_config.yaml                # map users → tabs (and display names)
├── lib/
│   ├── kafka_producer.py              # thin wrapper around Kafka producer
│   └── spotify_payloads.py            # helpers for common Spotify transforms
├── docker/                            # build contexts (app/consumer/producer
images)
│   ├── app.Dockerfile
│   ├── consumer.Dockerfile
│   └── producer.Dockerfile
├── tools/
│   └── install_docker_ubuntu.sh
└── docs, notebooks, ...

```

Ports (by default): Streamlit on <http://localhost:8501>, Mongo Express on <http://localhost:8081> (see [src/docker-compose.yml](#)).

3) Naming conventions (follow exactly)

Let **<prefix>** be your short ID (e.g., **alex**, **gilian**, **vadim**).

Kafka topics

- Two base topics are pre-wired by convention (can be extended):
 - `${APP_PREFIX}_recent_events`**
 - `${APP_PREFIX}_artist_market_top_tracks`**
- You may define **your own topics** in your `.env.<prefix>` (comma-separated list in **`KAFKA_TOPICS`**).
Use only lowercase, digits and underscores. Do **not** mix dot `.` and underscore `_` in the same project.

MongoDB collections

- Default collections used by consumer/Streamlit are set in `.env`:
 - `MONGO_COLL_EVENTS=${APP_PREFIX}_recent_events`**
 - `MONGO_COLL_TOP10=${APP_PREFIX}_artist_market_top_tracks`**
- If you add **new topics**, either:
 - Reuse the **generic mapping** (collection name equals the topic name), or
 - Point your consumer/Streamlit to explicit collections via envs or code.

Files and modules

- Producer module:** `src/producers/<prefix>_producer.py` (export a `main()` or runnable module via `python -m producers.<prefix>_producer`).
- Streamlit tab:** `src/app/tabs/<prefix>.py` (expose a function `render()` or follow the existing tab style).
- User env:** `src/envs/.env.<prefix>` (copied into `.env` before running).

4) Your producer — where to put what

1. **Create your module** by copying the example:

```
cp src/producers/example_producer.py
src/producers/<prefix>_producer.py
```

2. Implement your Spotify logic inside `<prefix>_producer.py`:
 - Read credentials and config from `os.environ` (already in place in example).
 - Produce JSON **per record** to one of your topics from `KAFKA_TOPICS`.
 - Use the helper `lib/kafka_producer.py` (already imported in example) or standard `kafka-python/confluent_kafka` if you prefer.
3. **Schema differences are OK**. Spark and Streamlit tabs are written to tolerate different payloads; for tab-specific visuals, handle your own fields safely (use `.get()` with defaults).

The container command that runs producers is:

```
python -m producers.${PRODUCER_ENTRY} (from src/docker/producer.Dockerfile and
docker-compose.yml).
```

We control `PRODUCER_ENTRY` from `.env` (see next section).

5) Your .env — one per teammate

Create `src/envs/.env.<prefix>` by copying Dorin's reference and changing values:

```
# =====
# <PREFIX> – Docker-only environment configuration
# =====
APP_PREFIX=<prefix>

# Spotify API (your own app credentials)
CLIENT_ID=...
CLIENT_SECRET=...
USERNAME=<your_spotify_username_or_id>
REDIRECT_URI=http://127.0.0.1:8888/callback
MARKET_OVERRIDE=DE

# Spotipy (keep consistent with above)
SPOTIPY_CLIENT_ID=${CLIENT_ID}
SPOTIPY_CLIENT_SECRET=${CLIENT_SECRET}
SPOTIPY_REDIRECT_URI=${REDIRECT_URI}

# Debug / feature flags
DEBUG=true
KAFKA_ENABLED=true

# Kafka (inside Docker network)
KAFKA_BOOTSTRAP=kafka:19092
```

```
# Your topic(s) (comma-separated). Use your prefix in names:
KAFKA_TOPICS=${APP_PREFIX}_recent_events,${APP_PREFIX}_artist_market_top_tracks

# Optional routing hint (not mandatory)
KAFKA_TOPIC_ROUTES=${APP_PREFIX}_recent_events:events,${APP_PREFIX}_artist_market_top_tracks:top10

# Mongo (inside Docker network)
MONGO_URL=mongodb://root:example@mongo:27017/?authSource=admin
MONGO_DB=spotify_db
MONGO_COLL_EVENTS=${APP_PREFIX}_recent_events
MONGO_COLL_TOP10=${APP_PREFIX}_artist_market_top_tracks
GENERIC_COLL_NAMESPACE=${APP_PREFIX}__topic__

# Which producer module to run in the producer container
PRODUCER_ENTRY=<prefix>_producer

# Spark (only if you need to tweak; safe defaults below)
SPARK_KAFKA_TOPICS=
SPARK_TRIGGER_SECS=10
SPARK_WATERMARK_MIN=5
SPARK_MODE=top_artists
SPARK_TOPN=10
SPARK_FEATURE=track_duration_ms
SPARK_GROUP=market_used
SPARK_PACKAGES=org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1,org.apache.kafka:kafka-clients:3.5.2

# Streamlit (host port)
PORT=8501
```

Load your env into **.env** with the Makefile target (see §7):

make <prefix>-env → this copies **src/envs/.env.<prefix>** to **src/.env**.

Tip: Keep secrets locally. **.env** and real **.env.<prefix>** are git-ignored. Only commit example files like **.env.<prefix>.example** if you want to share structure.

6) Streamlit — add your own tab

1. Create **src/app/tabs/<prefix>.py**. Start from an existing tab (e.g. **alex.py** or **avd.py**).
2. **Do not break the app**: your tab must not crash if data is missing. Use tolerant reads with **.get()** and guard empty states.
3. Register your tab in **src/app/team_config.yaml**:

```
users:
  - id: alex
    name: "Alex"
    tab: "alex"          # file src/app/tabs/alex.py
  - id: <prefix>
```

```
name: "<Your Name>"
tab: "<prefix>"      # file src/app/tabs/<prefix>.py
```

4. Rebuild **app** container or let Compose rebuild on change:

```
make app-build && make app-up
# open http://localhost:8501
```

7) Makefile targets you will use (from **src/makefile**)

- **Environment (copy your env into **.env**):**

```
cd src
make <prefix>-env      # we keep <prefix>-env targets per teammate
make show-env          # quick check of key vars loaded into .env
```

If **<prefix>-env** doesn't exist yet in the main Makefile, add it like this:

```
<prefix>-env:
    @cp envs/.env.<prefix> .env
    @echo "[ENV] Loaded .env.<prefix> for <Your Name>"
```

Or create your helper in **make/_template.mk** and include it as needed.

- **Bring up infra (Zookeeper, Kafka, Mongo, App, etc.):**

```
make up
make logs      # tail all service logs
```

- **Kafka topics from your **.env**:**

```
make kafka-init-from-env
make kafka-list
# tail messages from a topic:
make kafka-tail-topic TOPIC=<your_topic>
```

- **Run your producer (inside container, uses **PRODUCER_ENTRY**):**

```
make producer-build
make producer-run
```

```
make producer-logs
```

- **Spark streaming job & logs:**

```
make spark-build
make spark-up
make spark-logs
```

- **Streamlit app:**

```
make app-build
make app-up
make app-logs
# open http://localhost:8501
```

- **Down / cleanup (containers only; volumes remain):**

```
make down
```

- **One-shot per-user demo (if defined):** If available (see Dorin's `avd-demo` target):

```
make <prefix>-demo
```

It performs: `infra up` → `kafka-init-from-env` → run your producer once → start Spark + Streamlit.

8) Consumer behavior (Kafka → Mongo)

- Default consumer `src/consumers/kafka_consumer.py` reads any topics you define in `KAFKA_TOPICS` and writes to:
 - explicitly configured collections (`MONGO_COLL_*`) **or**
 - a **generic** collection per topic (fallback), using your topic name (or `GENERIC_COLL_NAMESPACE` if set).
- If your payload schema differs (it will!), the consumer stores the raw JSON as-is. No schema enforcement here; do schema-aware queries in your tab or through Spark.

If you need custom projection/transforms, either:

- fork the consumer (create `consumers/<prefix>_consumer.py` + compose a new service),
or
- post-process in Spark and read from Spark output collections in your tab.

9) Spark streaming

- `spark/streaming_job.py` reads Kafka topics (from `.env`) and writes aggregations to Mongo.
 - Modes available (via `SPARK_MODE`): `top_artists`, `top_tracks`, `feature_avg` (with `SPARK_FEATURE` and `SPARK_GROUP`).
 - For **custom schemas**, add parsing logic in `spark/schemas.py` (or within the job) and wire your topics with your own branch in `streaming_job.py`.
 - Packages required are set in env (`SPARK_PACKAGES`). No need to edit the container unless you add exotic connectors.
-

10) Docker Compose services (what gets started)

- **zookeeper**, **kafka**, **mongo**, **mongo-express** (port usually `8081` → check compose), **producer**, **consumer**, **spark-app**, **streamlit-app**.
 - All services use the **same** `.env` at `src/.env` (copied from your `src/envs/.env.<prefix>`).
-

11) Quick Start — checklist for a new teammate

1. **Get Docker** (Ubuntu users can run `src/tools/install_docker_ubuntu.sh`).
2. Clone repo, then:

```
cd src
cp envs/.env.avd envs/.env.<prefix>      # edit with your Spotify
creds + topics
make <prefix>-env                        # copies to .env
make show-env
```

3. Add your producer:

```
cp producers/example_producer.py producers/<prefix>_producer.py
# implement your logic
```

4. Register your tab (optional but recommended):

```
cp app/tabs/alex.py app/tabs/<prefix>.py
# edit; then add entry to app/team_config.yaml
```

5. Build & run:

```
make up
make kafka-init-from-env
make producer-build && make producer-run
```

```
make spark-build && make spark-up
make app-build && make app-up
# Open UI: http://localhost:8501
# Mongo Express (if enabled in compose): http://localhost:8081
```

12) Troubleshooting

- **Port already in use** (e.g., 9092/2181/27017/8081/8501):
 - Stop old containers: `docker ps -a`, then `docker stop <id>` and `docker rm <id>`.
 - Find blocker: `sudo ss -ltnp '( sport = :PORT )'` then kill PID or change host port in `docker-compose.yml`.
 - Recreate service: `docker compose up -d --force-recreate <service>`.
- **Kafka not ready**: run `make kafka-wait` (built into `kafka-init-from-env`).
- **Auth to Spotify**: first run opens a local auth server; ensure redirect URI matches.
- **Windows/WSL**: run commands inside the repo folder in WSL; Docker Desktop must be running.
- **Docker Desktop**: if builds fail, restart Docker Desktop and retry `make ...-build`.

13) What to commit (and what NOT)

- **Commit**: your code in `src/producers/<prefix>_producer.py`, `src/app/tabs/<prefix>.py`, any Spark logic updates, **example env** `src/envs/.env.<prefix>.example` (no secrets).
- **Do NOT commit**: real `.env` or any file with secrets (`.env`, token caches, client secrets). The repo `.gitignore` is set properly.

14) Note

- There is a ready-to-use Ubuntu installer: `src/tools/install_docker_ubuntu.sh`.
- The Makefile is Docker-only and safe to use on any teammate machine.
- If you're stuck, ask in the group; please **test this weekend** so we can ship on time.